CNCF Project Focus

Episode #7

HAMi

Heterogeneous GPU sharing on Kubernetes

Christian Dussol

Read more

Cloud Native AI Infrastructure

Three questions behind running AI workloads on Kubernetes

1

How do workloads share a GPU ?

This episode

2

How do workloads get their data ?

Next episode

3

When is a workload admitted ?

Closing episode

One pod, one whole GPU

Kubernetes counts **nvidia.com/gpu** as whole devices

Example: a pod that needs 4000 MiB still gets the entire GPU

```
resources:
  limits:
    nvidia.com/gpu: 1
```

Who needs a slice of a GPU

Small-model inference

GOOD FIT

A model of a few GB serving a modest traffic leaves most of the GPU idle.

Notebooks, dev and CI

GOOD FIT

Experiments and tests need GPU time for minutes, not a whole GPU for days.

Large-model serving

IT DEPENDS

vLLM targets 90% of GPU memory by default.
Size it before sharing.

Large-model training

POOR FIT

A job that keeps every compute unit busy: sharing only splits the GPU.

Sharing pays off when a workload uses only a fraction of the GPU

Ten pods, Ten GPUs

Ten pods, each with the same small request, **each** locking a **whole** GPU
You pay for ten GPUs, not for the memory your workloads **actually** use

What's inside a GPU

Three parts to share. HAMi **caps** two of them.

Compute units (SMs) — run the calculations in parallel

L2 cache — shared by every workload on the GPU

↕ memory bandwidth ↕

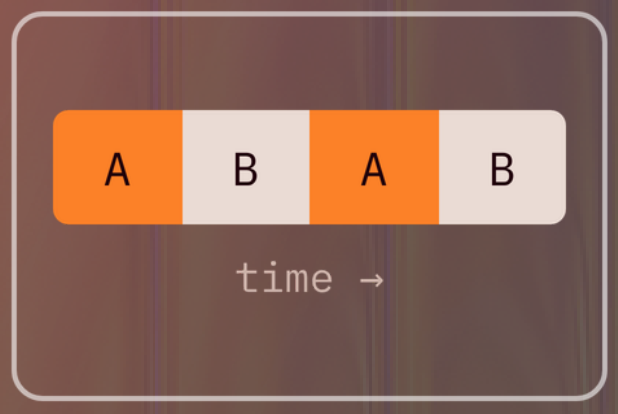
Memory (VRAM) — `15360 MiB on a T4`

holds the model weights and the data

LIMITED BY HAMi — Memory `gpumem` · Compute share `gpucores`

NOT PARTITIONED — L2 cache · Memory bandwidth

Three ways to share a GPU



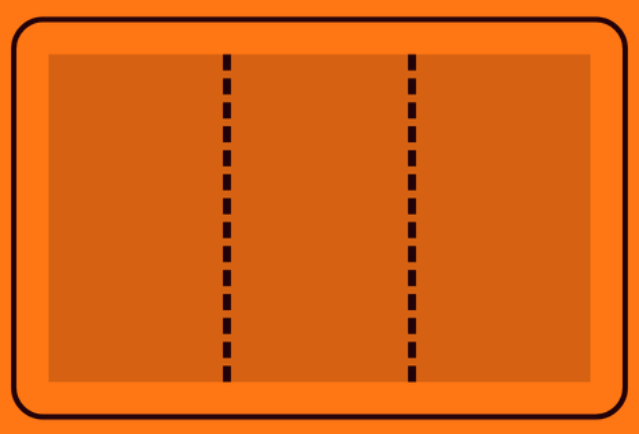
Time-slicing

Pods take turns on the GPU.
Nothing stops one of them from using all the memory

MIG

The GPU is physically split into partitions, each with its own memory and cores.

Available only on selected NVIDIA data-center GPUs.



Software partitioning

A library inside each container enforces memory and compute limits.

Cache and memory bandwidth are not partitioned.

What is HAMi ?

Heterogeneous AI computing virtualization middleware

An open source platform for managing heterogeneous AI computing devices in Kubernetes clusters

It enables device sharing across multiple containers and workloads

21 Aug 2024
Accepted into CNCF Sandbox

2 Jul 2026
Entered CNCF Incubating

v2.9.0
Version used in this episode's labs

Four components

1 **Mutating webhook** — ADMISSION

Sets schedulerName: hami-scheduler on GPU pods

2 **Scheduler extender** — CONTROL PLANE

Picks a GPU with room, writes the allocation

3 **Device plugin** — NODE

Registers the GPU, injects HAMi-core into the GPU container

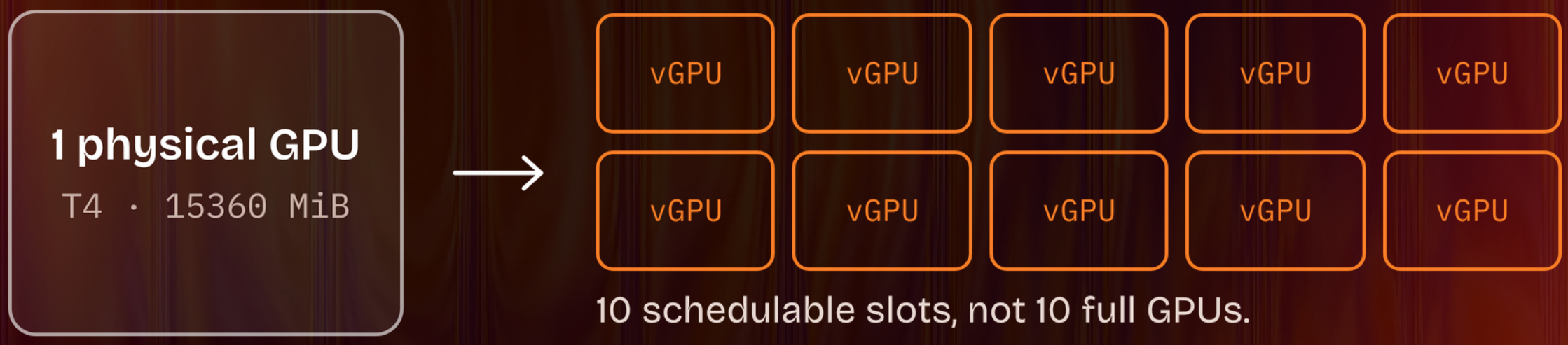
4 **HAMi-core** — CONTAINER

A library loaded into the container, enforces memory and compute limits

One GPU, ten vGPUs

EXPOSE → SHARE → LIMIT → MEASURE

The device plugin registers each physical GPU on the node, split into 10 schedulable slots by default.



NODE ALLOCATABLE

```
nvidia.com/gpu: 10
```

Source: official HAMi Lab 1 · project-hami.io/tutorials

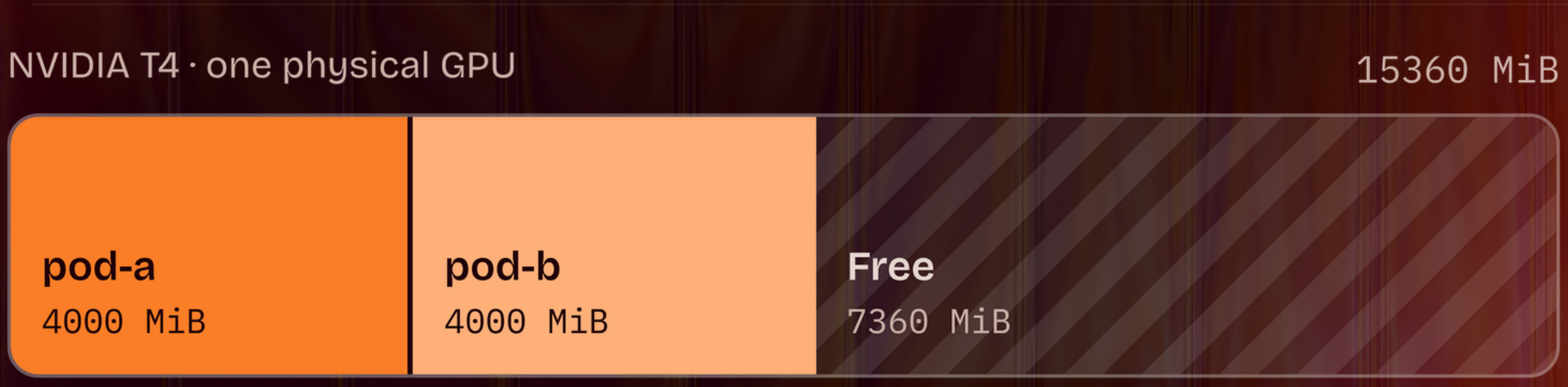
Request a slice

EXPOSE → SHARE → LIMIT → MEASURE

gpumem-pod-a

```
resources:
  limits:
    nvidia.com/gpu: 1
    nvidia.com/gpumem: 4000
```

Two pods with the same request land on the same physical GPU.



Source: official HAMi Lab 3 · project-hami.io/tutorials

The memory ceiling

EXPOSE → SHARE → LIMIT → MEASURE

WHAT THE CONTAINER IS TOLD · nvidia-smi

```
Tesla T4     0MiB / 4000MiB
```

HAMi-core rewrites the numbers. The physical GPU has 15360 MiB. The proof is the next test.

oom-test-pod · ALLOCATES 512 MiB AT A TIME

In this test, 7 chunks fit (3584 MiB). The 8th, plus the CUDA context, would cross the limit.

```
CUDA out of memory
```

- gpumem-pod-a, gpumem-pod-b: still Running

Source: official HAMi Lab 3 · project-hami.io/tutorials

The compute share

EXPOSE → SHARE → LIMIT → MEASURE

```
env:
  - name: GPU_CORE_UTILIZATION_POLICY
    value: "force"
resources:
  limits:
    nvidia.com/gpucores: 30    # + gpu, gpumem (slide 11)
```

DEFAULT POLICY

Up to 100%

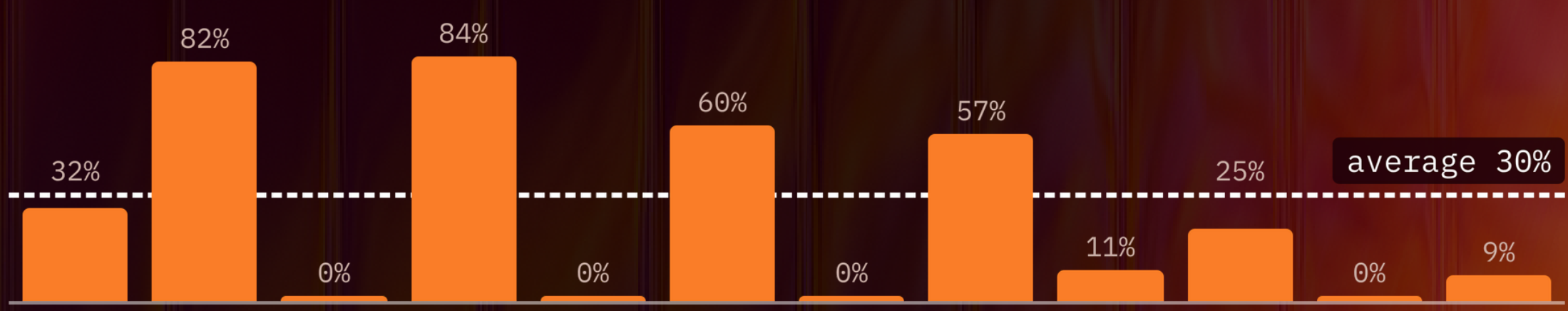
The pod can burst above its quota while the GPU is idle.

force

30%

Enforces the configured compute quota through duty cycling.

force · GPU UTILIZATION, 12 SAMPLES 5 s APART



Bursts of work, then enforced idle. Single samples go well above 30%; the average holds the quota.

The DRA path

The same slice, requested through Kubernetes-native Dynamic Resource Allocation

EXTENDED RESOURCES

- Mutating webhook
- ↓
- Scheduler extender
- ↓
- Device plugin

DRA

- ResourceClaim
- ↓
- kube-scheduler (native)
- ↓
- HAMi DRA driver

`ResourceClaim · single-gpu-0`

```
capacity:
 requests:
   cores: 30
   memory: "4Gi"
```

DRA replaces the scheduling path.
HAMi-core still enforces the limits in the container.

Try it
Official HAMi labs

Lab 2 · Local Fake GPU

START LOCALLY

Laptop, no GPU required

Lab 1, then Lab 3

SEE REAL SHARING

One cloud VM with a T4

Lab 4 · DRA Slicing

EXPLORE NATIVE DRA

Experimental, Kubernetes 1.34+, DRAConsumableCapacity feature gate

Interactive Manifest Generator

GENERATE YOUR MANIFEST

Pick a vendor, set memory and cores, get the YAML

project-hami.io/tutorials

LINK in the comments